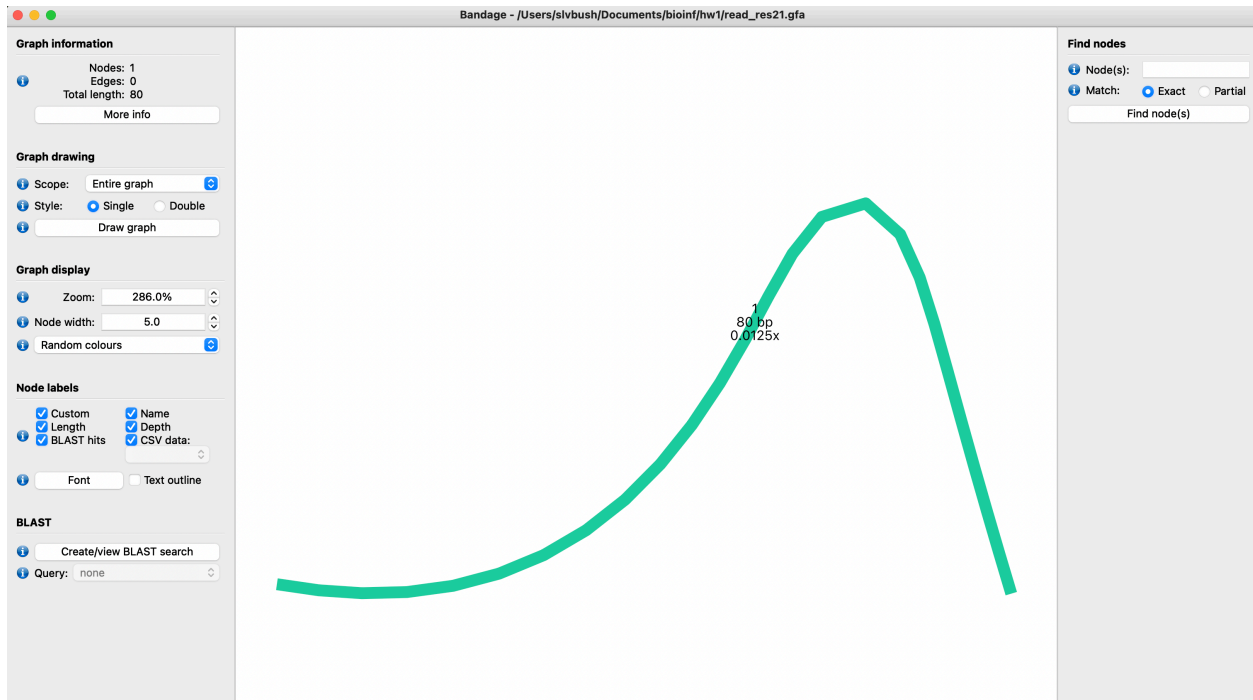


ОТЧЕТ

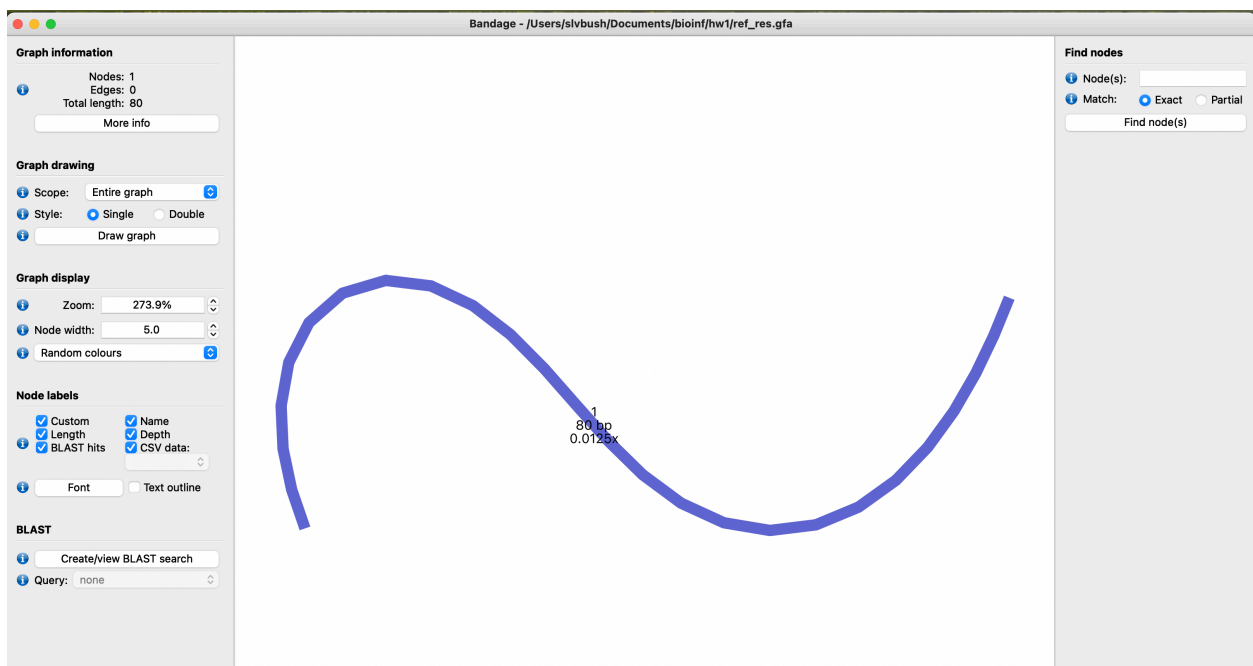
Скрины из Bandage:

ecoli_1k

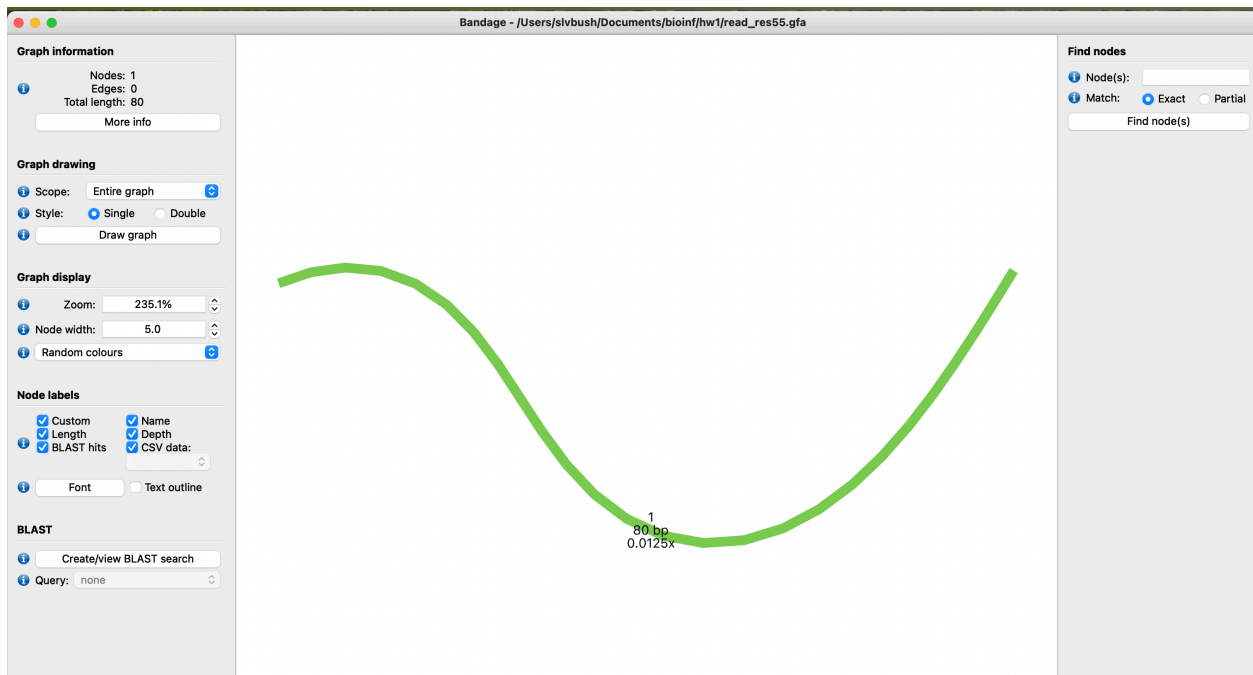
K = 21:



K = 31:

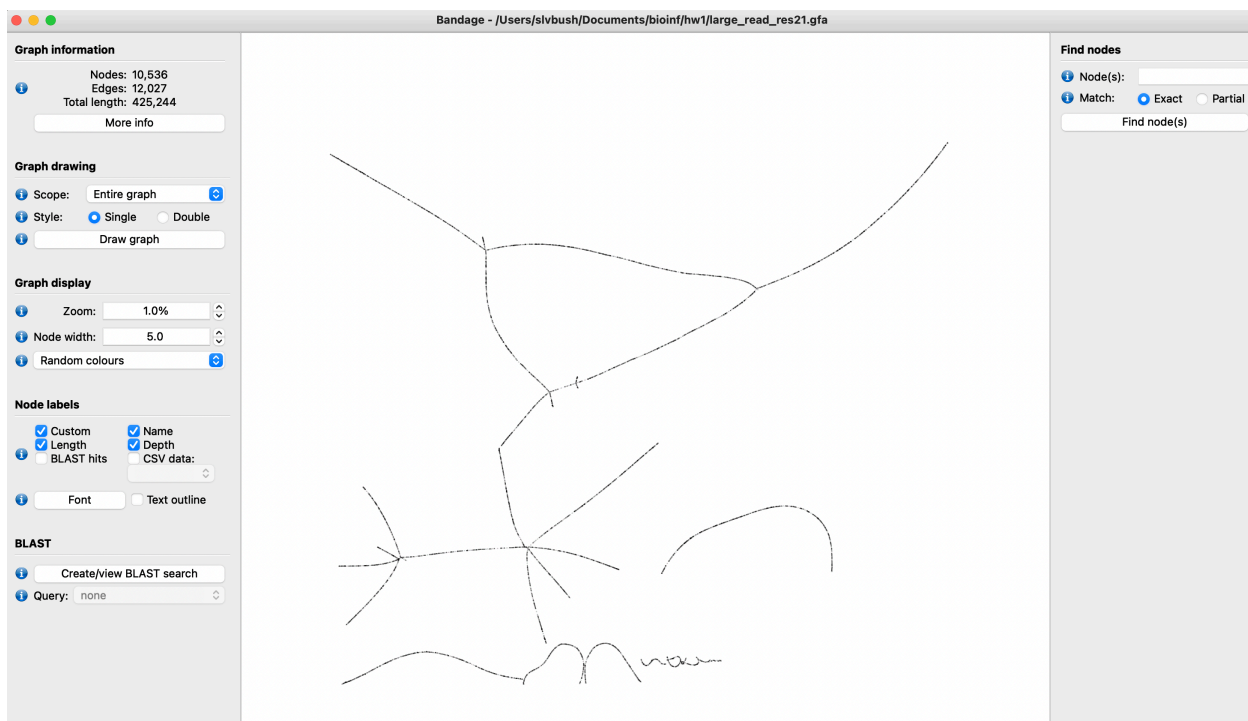


K = 55:

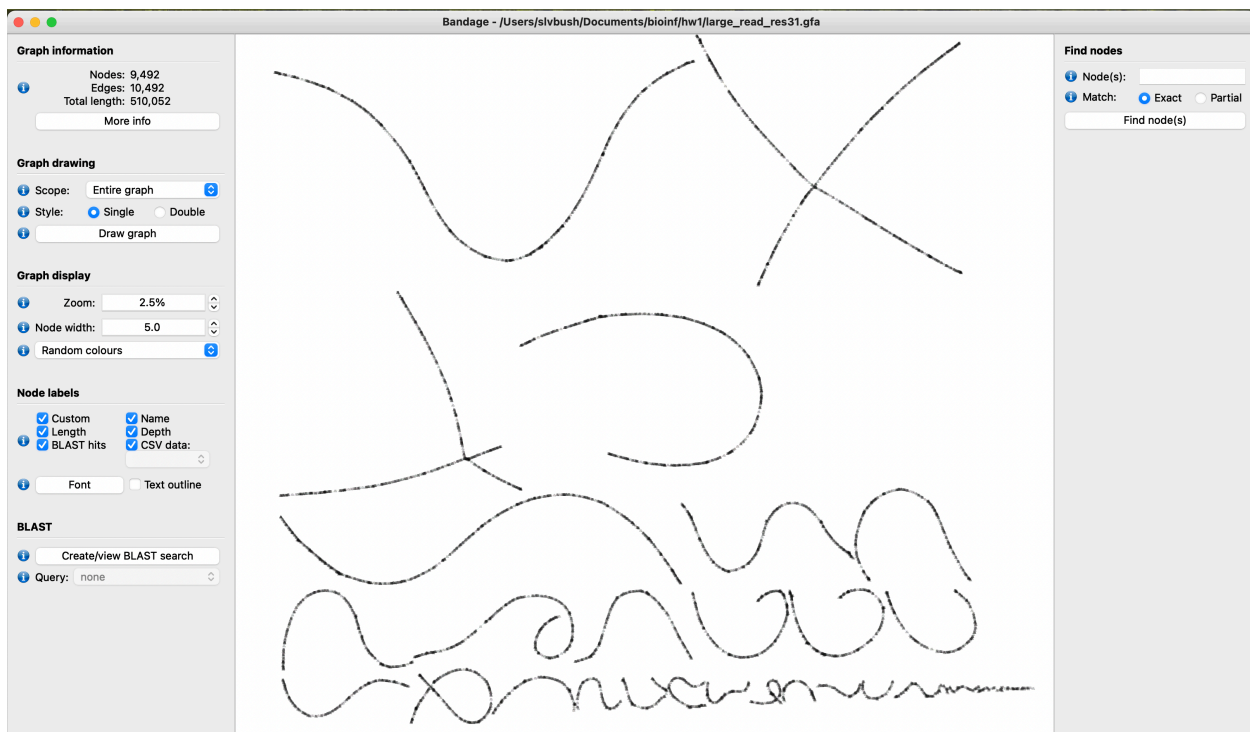
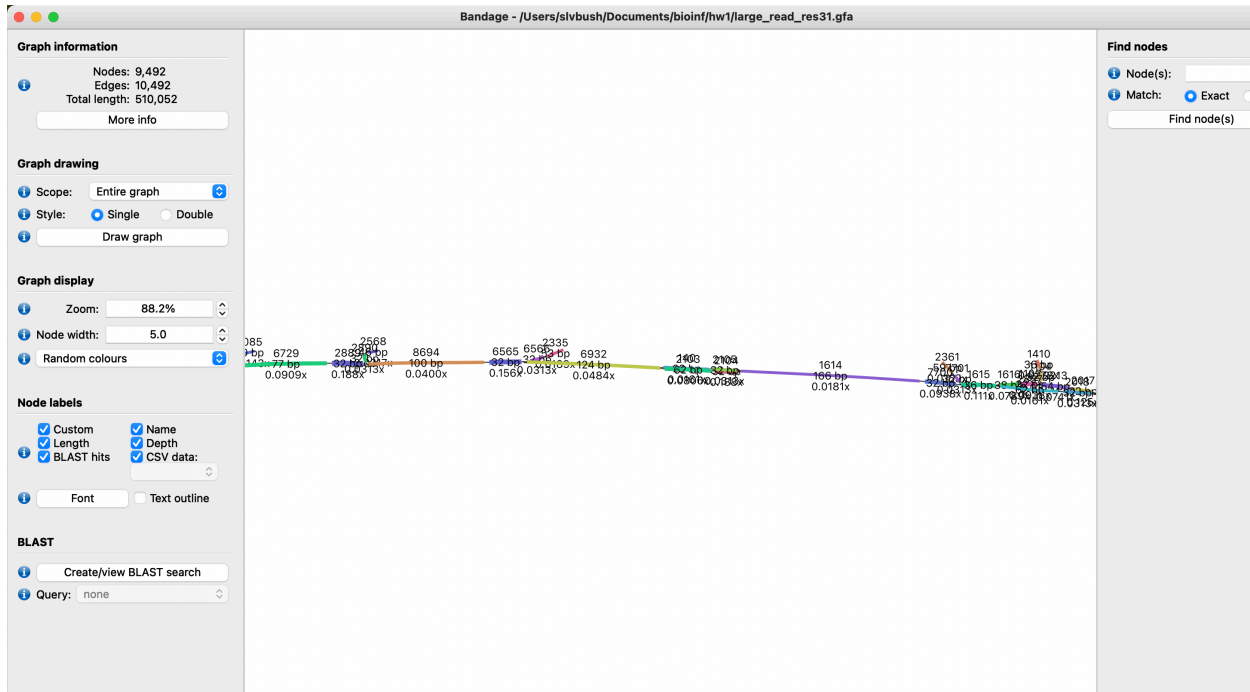


ecoli_reads.fastq

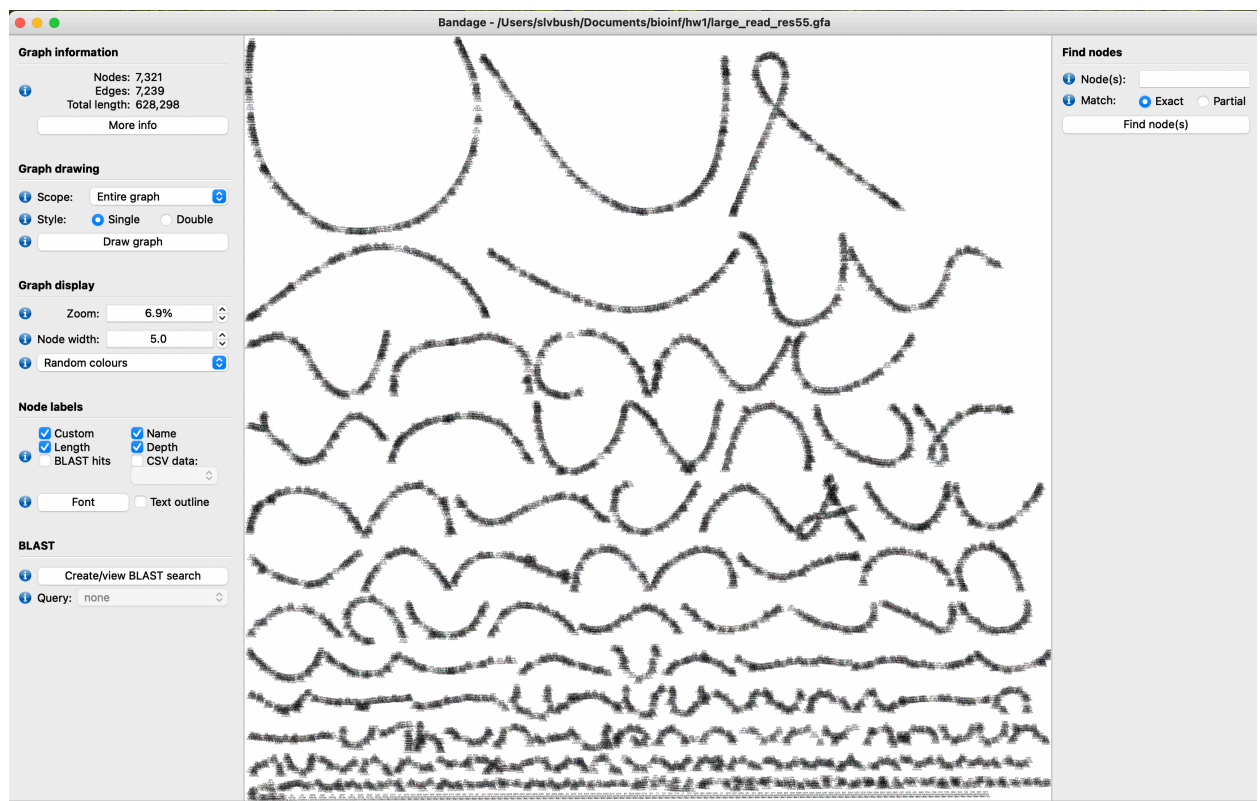
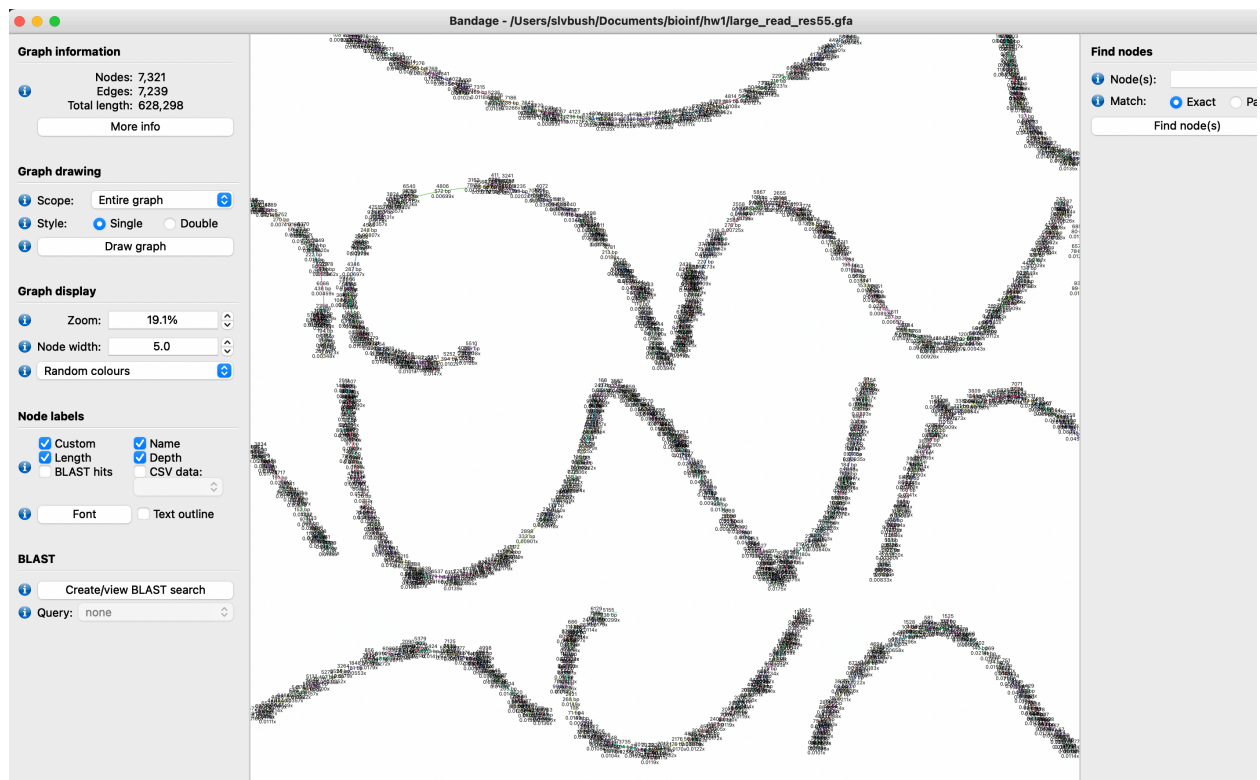
K = 21:



K = 31:



K=55:



Вывод по визуализации:

Результаты показывают, что программа работает правильно.

На образце 1k при любых k получается один короткий сегмент длиной 80. Это подтверждает, что алгоритм стабилен на чистых данных без ошибок.

На реальных чтениях reads картина меняется в зависимости от k (точные числа можно увидеть на картинках):

При $k = 21$ граф очень сложный, в нем больше 10 тысяч узлов и 12 тысяч ребер. Это происходит потому, что короткие k-меры не могут отличить уникальные участки от повторов, и получается запутанный клубок.

При $k = 55$ граф становится чище: узлов и ребер остается около 7 тысяч. Общая длина собранных последовательностей растет с 425 тысяч до 628 тысяч. Это значит, что длинные k-меры лучше соединяют фрагменты в целые цепочки и отсеивают мусор.

Итог: для сборки ридов лучше всего подошло $k = 55$, так как оно дает самые длинные контиги и наиболее понятный граф.

Важную роль в получении результатов сыграла очистка графа. Применение функции фильтрации по покрытию (`min_cov`) позволило удалить «мусорные» узлы и ребра, возникшие из-за ошибок секвенирования. Без очистки граф был бы перегружен мелкими тупиковыми ветками и ложными пузырями, которые мешают сборке длинных контигов.

Немного по коду (там есть комменты, но всё же):

В данной реализации граф де Брейна строится последовательным проходом по чтениям. Для каждой строки извлекаются все перекрывающиеся $(k+1)$ -меры: префикс длины k задаёт начальную вершину, суффикс — конечную. Граф хранится в

`unordered_map`, где каждой вершине соответствует список исходящих рёбер, а `in_degree` используется при сжатии.

Покрытие реализовано как счётчик повторяющихся переходов между вершинами. Если ребро между двумя вершинами уже существует, его `coverage` увеличивается, иначе создаётся новое. Таким образом, одинаковые рёбра агрегируются.

Очистка графа реализована через удаление рёбер с покрытием ниже заданного порога.

При этом корректируется `in_degree`, чтобы сохранить согласованность структуры.

Сжатие выполняется итеративно: объединяются линейные цепочки, где у вершины одно исходящее ребро, а следующая вершина имеет входящую степень 1 и тоже одно исходящее ребро. Последовательности склеиваются с учётом перекрытия длины k, покрытие усредняется, промежуточная вершина удаляется.

Сохранение в GFA реализовано через формирование сегментов (по одному на ребро) и связей между ними через общие вершины. В FASTA каждая последовательность на ребре записывается как отдельный `contig` с указанием длины и покрытия.

Парсер поддерживает FASTA и FASTQ: формат определяется по расширению, для FASTQ дополнительно пропускаются строки с качествами.

